Cloud Computing / Distributed Systems

Group Project: Build Your Own Distributed File System

1. Overview

In this project your team will design and implement your **own distributed file system** for text files, inspired by the architecture of the Google File System (GFS) studied in class. The goal is to apply the core ideas of partitioning (sharding), replication, and the separation of a metadata/naming layer from a storage layer to a working, runnable system.

The system must split files into fixed-size chunks, distribute those chunks across multiple storage servers, replicate them for fault tolerance, and expose a small set of operations through a client. Along the way you will reason about which failures your design can survive and which it cannot.

2. System Requirements

2.1 Architecture

- **Naming server (single):** indexes all files and knows where every chunk is stored. Acts as the metadata authority for the system.
- **Storage servers (multiple):** each stores only a **fraction** of the files' chunks — not the whole dataset.
- **Client:** a tool/library that makes the file system easy to use and hides the distribution from the end user.

2.2 Data handling

- The file system supports **text files only**.
- **Sharding:** files are split into chunks with a fixed **chunk size of 1 KB**.
- **Replication:** every chunk must be replicated across more than one storage server.
- You **may use a database** to store metadata, but the **actual chunk content must be stored in the file system** (not inside the database).

2.3 Supported operations

The client must support the following operations:

Operation	Expected behaviour
Create file	Accept a text file, split it into 1 KB chunks, distribute and replicate them across storage servers, and register the file in the naming server.
Read file	Look up chunk locations via the naming server, fetch all chunks (from any replica), reassemble, and return the original content.
Delete file	Remove the file's metadata and all of its chunk replicas from the storage servers.

Get size of file	Return the size of a stored file (e.g. from metadata) without transferring its full content.

3. Deliverables

Submit the following as the result of the project:

1. **Architecture document** outlining the system design, which failures are critical and which are not, and any other important design decisions and trade-offs.
2. **Dockerized applications** — the naming server, storage servers, and client packaged as containers.
3. **Instructions on how to run** the file system (e.g. docker-compose, environment, ports).
4. **Instructions on how to use** your system (the client operations, with examples).

All deliverables should live in a single Git repository with a clear README.

4. Fault-Tolerance Analysis

A core part of the grade is reasoning about failure. In your architecture document, explicitly address:

- What happens when **one storage server goes down**? Can clients still read and write?
- What happens when the **naming server goes down**? (Identify this as a critical / single point of failure if it is one.)
- How does **replication** protect data, and how many simultaneous failures can the system survive?
- Which failures are **recoverable** versus which lead to **data loss or unavailability**.